

Ex. No.: 6a)
Date: 26/02/25

FIRST COME FIRST SERVE

Aim:

To implement First-come First- serve (FCFS) scheduling technique

Algorithm:

1. Get the number of processes from the user.
2. Read the process name and burst time.
3. Calculate the total process time.
4. Calculate the total waiting time and total turnaround time for each process 5.
- Display the process name & burst time for each process. 6. Display the total waiting time, average waiting time, turnaround time

Program Code:

```
#include <stdio.h>
int main() {
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], wt[n], tat[n];
    printf("Enter burst times:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    wt[0] = 0;
    for (i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    for (i = 0; i < n; i++)
        tat[i] = bt[i] + wt[i];

    printf("\nProcess\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);

    return 0;
}
```


Sample Output:

Enter the number of process:

3

Enter the burst time of the processes:

24 3 3

Process	Burst Time	Waiting Time	Turn Around Time
0	24	0	24
1	3	24	27
2	3	27	30

Average waiting time is: 17.0

Average Turn around Time is: 19.0

Result:

Program Executed successfully

Ex. No.: 6b)
Date: 26/02/25

SHORTEST JOB FIRST

Aim:

To implement the Shortest Job First (SJF) scheduling technique

Algorithm:

1. Declare the structure and its elements.
2. Get number of processes as input from the user.
3. Read the process name, arrival time and burst time
4. Initialize waiting time, turnaround time & flag of read processes to zero.
5. Sort based on burst time of all processes in ascending order
6. Calculate the waiting time and turnaround time for each process.
7. Calculate the average waiting time and average turnaround time.
8. Display the results.

Program Code:

```
#include <stdio.h>

void sort(int p[], int bt[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (bt[i] > bt[j]) {
                int temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }
}

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int p[n], bt[n], wt[n], tat[n];
    for (int i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("Burst time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    sort(p, bt, n);
```

```
    wt[0] = 0;
    for (int i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];
    for (int i = 0; i < n; i++)
        tat[i] = bt[i] + wt[i];

    printf("\nProcess\tBT\tWT\tTAT\n");
    for (int i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);

    return 0;
}
```

Sample Output:

Enter the number of process:

4

Enter the burst time of the processes:

8 4 9 5

Process	Burst Time	Waiting Time	Turn Around Time
2	4	0	4
4	5	4	9
1	8	9	17
3	9	17	26

Average waiting time is: 7.5

Average Turn Around Time is: 13.0

Result:

Program Executed successfully

Ex. No.: 6c)
Date: 19/03/25

PRIORITY SCHEDULING

Aim:

To implement priority scheduling technique

Algorithm:

1. Get the number of processes from the user.
2. Read the process name, burst time and priority of process.
3. Sort based on burst time of all processes in ascending order based priority 4. Calculate the total waiting time and total turnaround time for each process 5. Display the process name & burst time for each process.
6. Display the total waiting time, average waiting time, turnaround time

Program Code:

```
#include <stdio.h>
```

```
void sort(int p[], int bt[], int pri[], int n) {  
    for (int i = 0; i < n - 1; i++)  
        for (int j = i + 1; j < n; j++)  
            if (pri[i] > pri[j]) {  
                int t = pri[i]; pri[i] = pri[j]; pri[j] = t;  
                t = bt[i]; bt[i] = bt[j]; bt[j] = t;  
                t = p[i]; p[i] = p[j]; p[j] = t;  
            }  
}
```

```
int main() {  
    int n;  
    printf("Enter number of processes: ");  
    scanf("%d", &n);  
  
    int p[n], bt[n], pri[n], wt[n], tat[n];  
    for (int i = 0; i < n; i++) {  
        p[i] = i + 1;  
        printf("Burst time for P%d: ", i + 1);  
        scanf("%d", &bt[i]);  
        printf("Priority for P%d (lower number = higher priority): ", i + 1);  
        scanf("%d", &pri[i]);  
    }  
}
```

```
sort(p, bt, pri, n);
```

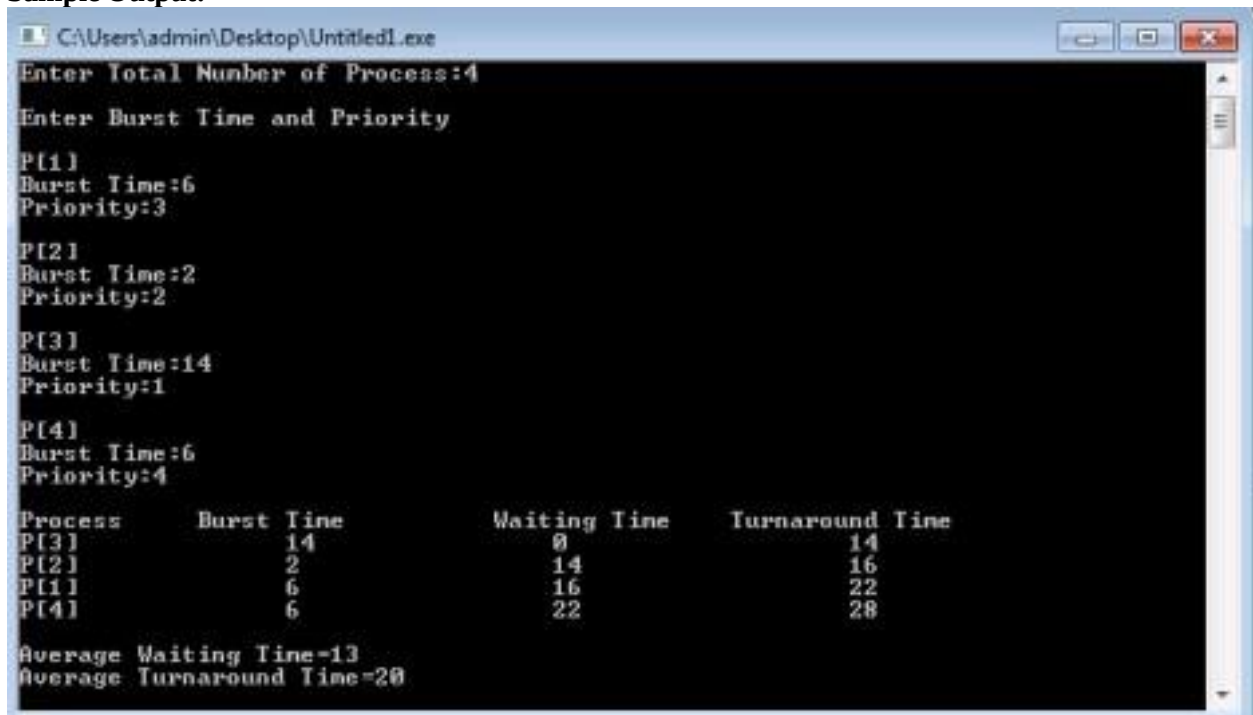
```
wt[0] = 0;  
for (int i = 1; i < n; i++)  
    wt[i] = wt[i - 1] + bt[i - 1];  
for (int i = 0; i < n; i++)
```

```
tat[i] = bt[i] + wt[i];

printf("\nProcess\tBT\tPriority\tWT\tTAT\n");
for (int i = 0; i < n; i++)
    printf("P%d\t%d\t%d\t\t%d\t%d\n", p[i], bt[i], pri[i], wt[i], tat[i]);

return 0;
}
```

Sample Output:



```
C:\Users\admin\Desktop\Untitled1.exe
Enter Total Number of Process:4
Enter Burst Time and Priority
P[1]
Burst Time:6
Priority:3
P[2]
Burst Time:2
Priority:2
P[3]
Burst Time:14
Priority:1
P[4]
Burst Time:6
Priority:4
Process      Burst Time      Waiting Time      Turnaround Time
P[3]          14              0                14
P[2]           2             14               16
P[1]           6             16               22
P[4]           6             22               28
Average Waiting Time=13
Average Turnaround Time=20
```

Result:

Program Executed successfully

Ex. No.: 6d)
Date: 19/03/25

ROUND ROBIN SCHEDULING

Aim:

To implement the Round Robin (RR) scheduling technique

Algorithm:

1. Declare the structure and its elements.
2. Get number of processes and Time quantum as input from the user.
3. Read the process name, arrival time and burst time
4. Create an array **rem_bt[]** to keep track of remaining burst time of processes which is initially copy of bt[] (burst times array)
5. Create another array **wt[]** to store waiting times of processes. Initialize this array as 0.
6. Initialize time : $t = 0$
7. Keep traversing the all processes while all processes are not done. Do following for i'th process if it is not done yet.
 - a- If $\text{rem_bt}[i] > \text{quantum}$
 - (i) $t = t + \text{quantum}$
 - (ii) $\text{bt_rem}[i] -= \text{quantum};$
 - b- Else // Last cycle for this process
 - (i) $t = t + \text{bt_rem}[i];$
 - (ii) $\text{wt}[i] = t - \text{bt}[i]$
 - (iii) $\text{bt_rem}[i] = 0;$ // This process is over
8. Calculate the waiting time and turnaround time for each process.
9. Calculate the average waiting time and average turnaround time.
10. Display the results.

Program Code:

```
#include <stdio.h>

int main() {
    int n, tq;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], rt[n], wt[n], tat[n];
    for (int i = 0; i < n; i++) {
        printf("Burst time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter time quantum: ");
    scanf("%d", &tq);

    int time = 0, done;
    do {
```

```

done = 1;
for (int i = 0; i < n; i++) {
    if (rt[i] > 0) {
        done = 0;
        if (rt[i] > tq) {
            time += tq;
            rt[i] -= tq;
        } else {
            time += rt[i];
            wt[i] = time - bt[i];
            rt[i] = 0;
        }
    }
}
} while (!done);

for (int i = 0; i < n; i++)
    tat[i] = bt[i] + wt[i];

printf("\nProcess\tBT\tWT\tTAT\n");
for (int i = 0; i < n; i++)
    printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);

return 0;
}

```

Sample Output

```
C:\WINDOWS\SYSTEM32\cmd.exe
Enter Total Number of Processes:      4

Enter Details of Process[1]
Arrival Time:  0
Burst Time:    4

Enter Details of Process[2]
Arrival Time:  1
Burst Time:    7

Enter Details of Process[3]
Arrival Time:  2
Burst Time:    5

Enter Details of Process[4]
Arrival Time:  3
Burst Time:    6

Enter Time Quantum:      3

Process ID      Burst Time      Turnaround Time      Waiting Time
Process[1]      4              13                   9
Process[3]      5              16                   11
Process[4]      6              18                   12
Process[2]      7              21                   14

Average Waiting Time:  11.500000
Avg Turnaround Time:  17.000000
```

Result:

Program executed successfully